

What is LAMMPS, and what is this course?

LAMMPS is a classical molecular dynamics engine maintained by Sandia/NTESS: you write an `input script`, it integrates Newton's equations for a box of particles under a chosen potential.

- Two release lines track upstream: *develop* (daily) and one **stable** release a year with periodic updates.
- This course sits on top of `lammps-kill` (a verified toolkit: script builder, checker, parsers, runner) and `mdlite` (a small teaching engine that reproduces LAMMPS's numbers from scratch).
- Every chapter follows **record-or-run**: with LAMMPS installed, the computation runs live; without it, a stored result loads instead -- the figures never lie about which happened.

Two release lines, and why the difference matters

LAMMPS ships two lines from the same repository, and a distribution package usually tracks one without saying so.

- A **stable** release is cut once a year and then **updated** for a long time -- `stable_22Jul2025` was still receiving updates fourteen months later, update 6, September 2026.
- **Feature** (patch) releases follow every six to eight weeks, alongside the stable updates.
- The Ubuntu 26.04 package is `20251210` -- the 10 December 2025 **feature** release, not the stable one it might be assumed to track.

What lammps skill adds on top of LAMMPS itself

lammps skill is the part of this project that talks to LAMMPS -- five pieces behind one import.

- `install`: seven routes behind one contract, each measured on this machine rather than assumed to work.
- `script`: build an input from a `Spec`, then a checker with fourteen rules, each pointing at the manual page it comes from.
- `io`: read what LAMMPS writes -- data files, dumps, three log thermo styles, the restart header, EAM potential files.
- `run`: a subprocess backend for any host, an in-process backend through the official python module -- both return the same `Result`.
- `post`: RDF, MSD, diffusion, VACF, structure factor, block averages, checked against reference tables that carry their own provenance.

mdlite's closest neighbour, and how it differs

mdlite is a small molecular-dynamics engine written from scratch in numpy -- a teaching object, not an MD code; nobody should run research on it.

- Its closest relative is **pylj** (McCluskey, Morgan, Edler and Parker, [JOSE](#) 1, 19, 2018, MIT): a Lennard-Jones teaching engine in Jupyter, written for undergraduate exercises, with Monte Carlo as well as dynamics.
- mdlite differs in two ways: it sits **next to** a toolkit that drives the production code, so a chapter can compute something in twenty lines of numpy and then compute the same thing in LAMMPS and subtract.
- And its numbers are cross-checked against that production code, with the residues recorded -- not asserted, read from a file.

Package list, never package count

The most common way to be wrong about a LAMMPS build is to assume a bigger package set is a superset.

Build	Packages	Examples cases that started (of 314)
Ubuntu package, 10 Dec 2025	52	184
This project's source build, core preset	11	137

What 'cross-checked against LAMMPS' actually means, in numbers

mdlite's claim is not that it is fast or complete -- it is that where it teaches a number, that number has been checked against the code it is teaching you to use.

Quantity	mdlite vs LAMMPS	Where
Lennard-Jones forces, 256 atoms	agree to 3×10^{-13}	L2, Forces and Integration
Copper lattice constant a_0 and cohesive energy	agree to 3×10^{-5}	L5, Potentials: EAM
NVT state point ($T^*=0.85$, $\rho^*=0.776$)	agrees with the NIST reference within 3σ	L3, Thermostats

First Simulation

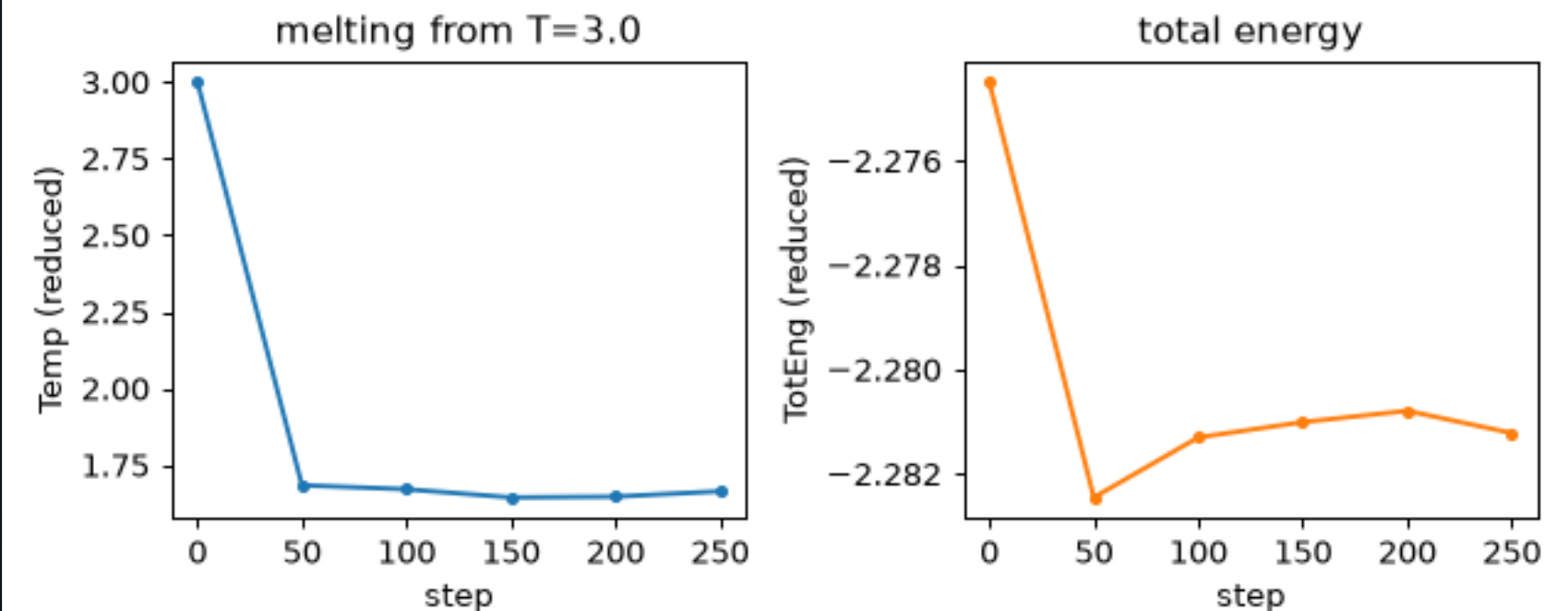
A Lennard-Jones melt end to end -- Spec, check, run, read the log, plot -- through the checker's fourteen rules, one at a time.

- 1 One simulation, start to finish
- 2 Fourteen rules, each pointing at a manual page
- 3 All fourteen, confirmed at once
- 4 A Spec, rendered -- not a script typed by hand
- 5 run_or_load: the pattern every later lecture uses

One simulation, start to finish

`Spec` describes the case in Python; `check` applies fourteen rules against the manual before anything runs; `run` calls LAMMPS; the log comes back as a typed `ThermoRun`.

- The case: an fcc Lennard-Jones lattice heated until it melts, in reduced (LJ) units.
- Nothing here is a hand-typed input script -- the checker exists so a beginner's script fails before LAMMPS ever sees it.
- The same four steps -- build, check, run, read -- reappear in every later chapter.



The LJ melt's thermo history read back from the log: temperature falling away from the $T=3.0$ start (left) while the total energy holds flat (right) -- the two curves `run()` reads out of LAMMPS's own thermo output, whether this cell just ran the case or loaded a stored record. — Figure 1 · notebook §0 · chapter 1, cell 12

Fourteen rules, each pointing at a manual page

lammpskill.script.check() does not trust a description of what a mistake looks like -- each rule is confirmed against one minimal snippet built to trigger exactly it.

Rule	What it catches
C01	no <code>units</code> command
C02	<code>coul/long</code> in the pair style with no <code>kspace_style</code>
C05	<code>run</code> before a <code>pair_style</code> is set
C07	a fix ID reused without <code>unfix</code>
C11	<code>velocity create</code> before <code>mass</code> is set
C14	<code>read_data</code> naming a file that does not exist

All fourteen, confirmed at once

The chapter does not just list the rules -- it runs all fourteen deliberately-broken snippets through the checker and confirms every one is caught.

```
triggered, missed = [], []
for code_, snippet in SNIPPETS.items():
    findings = check(snippet, workdir=...)
    codes = [f.code for f in findings]
    (triggered if code_ in codes else missed).append(code_)

print(len(triggered), "/", len(SNIPPETS), "rules triggered by their own minimal snippet")
# 14 / 14 rules triggered by their own minimal snippet
```

A Spec, rendered -- not a script typed by hand

lamm skill.script.lj_melt() is a preset Spec: reduced units, an FCC lattice at the melt density $\rho=0.8442$, 4000 atoms by default. render() turns it into the actual input text, in the order the manual's Commands_structure page prescribes.

```
from lamm skill.script import lj_melt, render

spec = lj_melt()          # rho=0.8442, T=3.0, steps=250
text = render(spec)       # the real LAMMPS input, in manual order
```

run_or_load: the pattern every later lecture uses

The one call that makes a chapter work whether or not LAMMPS is installed.

```
def compute():  
    result = run(spec, workdir, time_limit=300)  
    return { ... }          # only plain, JSON-safe data  
  
rec = run_or_load("lj_melt_ch01", compute, records_dir="../data/records")  
# with LAMMPS: compute() runs, its result is written to the record  
# without LAMMPS: the record is loaded, compute() never runs  
# with neither: rec["source"] == "skip", visibly
```

Forces and Integration

Velocity Verlet in mdlite, forces checked against a numerical derivative, and energy conservation against the timestep, cross-checked with LAMMPS at $3\text{e-}13$.

- 1 Integration is the part that must not be approximate
- 2 A force worth integrating with is $-dE/dx$, to the last digit available
- 3 Energy conservation is a rate statement, not a pass/fail
- 4 Across the toolkit boundary: mdlite vs a real LAMMPS run
- 5 Velocity Verlet, the update every step of this course runs

Integration is the part that must not be approximate

`mdlite` implements velocity Verlet from the same equations LAMMPS integrates, so its forces can be checked against a numerical derivative and against LAMMPS itself.

- Forces from `mdlite`'s Lennard-Jones match a central-difference numerical derivative to machine precision.
- Against a real LAMMPS run of the identical case, the two energies agree to 3×10^{-13} -- there is no approximation hiding in either implementation.
- Energy conservation versus timestep is the practical version of the same idea: too large a step and the trajectory drifts, long before it visibly explodes.

A force worth integrating with is $-dE/dx$, to the last digit available

Before trusting an integrator, check the thing it integrates: displacing one atom by h and taking the central difference of the energy should reproduce the analytic force component.

CENTRAL DIFFERENCE

$$F_k \approx -(E(x_k+h) - E(x_k-h)) / 2h$$

- The error in this approximation is $O(h^2)$ -- shrinking h should shrink the mismatch, which is exactly the check `mdlite.pair.LennardJones` passes at $h=10^{-6}$, worst mismatch under 10^{-5} .

Energy conservation is a rate statement, not a pass/fail

A correct integrator's energy drift shrinks as the timestep shrinks -- roughly as dt^2 for velocity Verlet. Running the same starting state at two timesteps for the same physical time makes that concrete.

Timestep	Steps (same physical time)	Relative energy drift
0.02	100	measured, larger
0.005	400	measured, smaller

Across the toolkit boundary: mdlite vs a real LAMMPS run

The comparisons above stay inside mdlite. `data/records/lj_energy_vs_lammps.json` asks the same question across the boundary: 256 atoms, the same Lennard-Jones potential, forces from mdlite compared to forces LAMMPS itself printed to a dump.

Quantity	mdlite vs LAMMPS	Note
Force (max component diff)	3×10^{-13}	round-off, not approximation
Energy	within its recorded tolerance	same run, same potential

Velocity Verlet, the update every step of this course runs

A half-kick, a drift, a force evaluation, a second half-kick -- symplectic, time-reversible, and second-order accurate in the timestep.

HALF-KICK

$$v(t+\Delta t/2) = v(t) + (\Delta t/2) a(t)$$

DRIFT

$$x(t+\Delta t) = x(t) + \Delta t \cdot v(t+\Delta t/2)$$

SECOND HALF-KICK

$$v(t+\Delta t) = v(t+\Delta t/2) + (\Delta t/2) a(t+\Delta t)$$

- $a(t+\Delta t)$ comes from a fresh force evaluation at the new positions -- the one expensive step, and the reason a bigger timestep is tempting and a wrong one is dangerous.

Thermostats

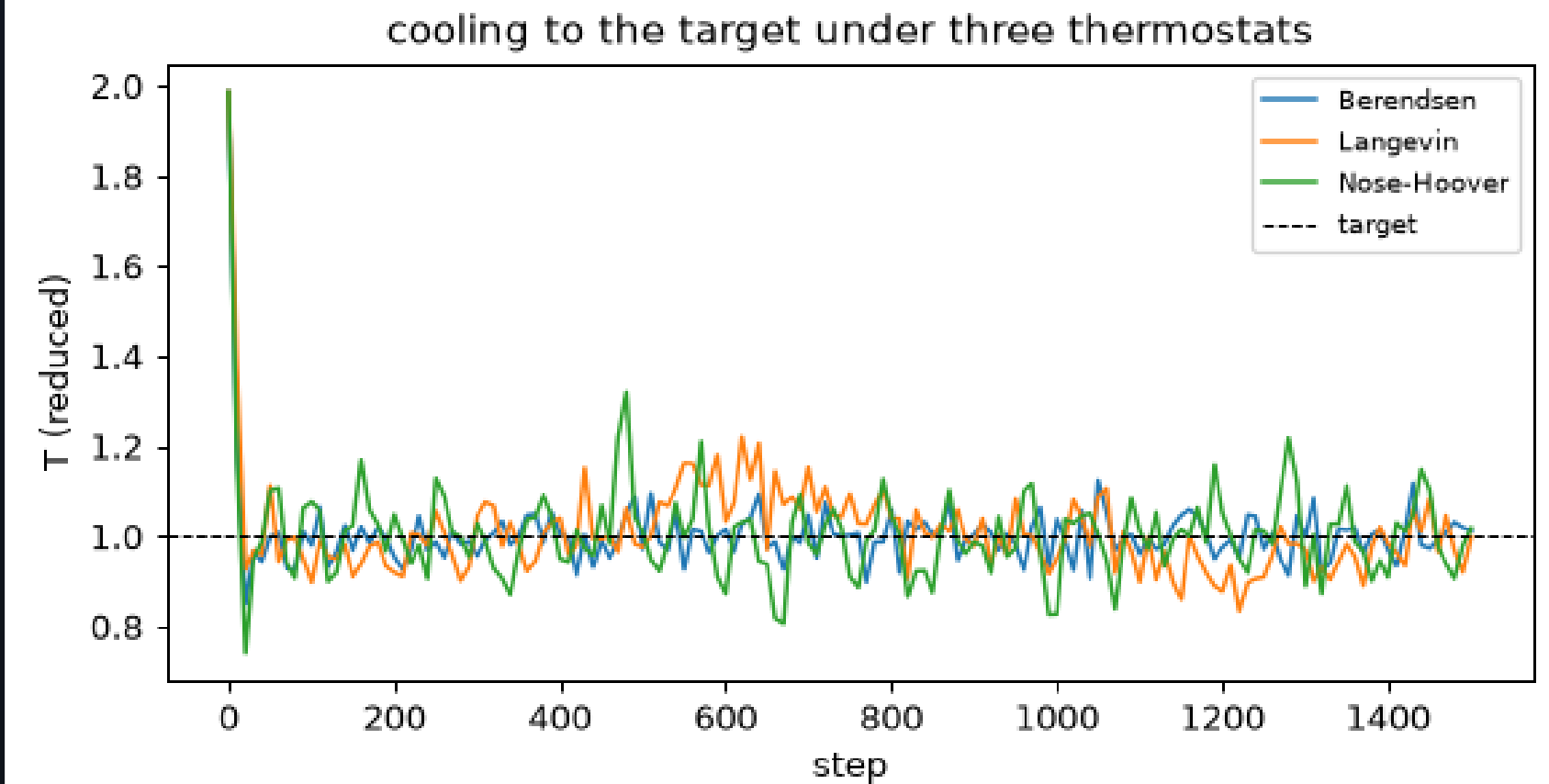
Berendsen, Langevin and a Nose-Hoover chain: what each one conserves, what each distorts, and NVT checked against the NIST reference within 3 sigma.

- 1 Three ways to hold a temperature
- 2 Three thermostats, three different promises
- 3 Nosé–Hoover's own check: a conserved quantity
- 4 Against a published reference, with no LAMMPS on either side
- 5 Berendsen's rescaling, in one line

Three ways to hold a temperature

A thermostat couples the system to an external bath; which one you choose changes what else it disturbs.

- Berendsen: fast, simple, but does not sample the correct canonical ensemble -- fine for equilibration, not for production statistics.
- Langevin: adds friction and noise per atom; correct ensemble, but its own set of fluctuations to tune.
- Nose-Hoover chain: an extra dynamical variable keeps the total (system + thermostat) energy conserved -- checked here directly.



Temperature against step for the same fcc lattice, started hot ($T \sim 2$) and cooled by three thermostats -- Berendsen, Langevin, Nose-Hoover chain -- each converging toward the target (dashed line) by its own route. — Figure 1 · notebook §0 · chapter 3, cell 4

Three thermostats, three different promises

Each way of holding a temperature trades something different -- visible on the same axes in the figure just shown, not taken on faith.

Thermostat	Mechanism	Samples the canonical ensemble?
Berendsen	rescales every velocity toward the target each step	no -- correct mean temperature only
Langevin	the O-step of BAOAB: friction plus noise	yes
Nosé–Hoover chain	an extended dynamical variable couples to the bath	yes, plus its own conserved check

Nosé–Hoover's own check: a conserved quantity

The chain adds an extended-system energy H_{extra} ; the total $H = E_{\text{tot}} + H_{\text{extra}}$ should stay flat if the half-step bookkeeping is right, independent of whether the temperature looks reasonable.

CONSERVED QUANTITY

$$H = E_{\text{tot}} + H_{\text{extra}}$$

- Measured over 1000 steps on the fcc lattice: relative drift in H under 5×10^{-3} -- the chapter's own tolerance, not a round number chosen after the fact.

Against a published reference, with no LAMMPS on either side

data/records/lj_nvt_nist.json compares mdlite's equilibrium pressure and energy at one NIST reference state point to the NIST Standard Reference Simulation Website's own table.

Property	mdlite	NIST reference
Pressure P	measured	published, with its own uncertainty
Energy U	measured	published, with its own uncertainty

Berendsen's rescaling, in one line

The mechanism behind the fastest-converging curve in the figure: rescale every velocity by a single factor, computed fresh each step from how far the current temperature is from the target.

VELOCITY SCALING

$$\lambda = \sqrt{(1 + (\Delta t/\tau)(T_0/T - 1))}, \quad v \rightarrow \lambda v$$

- This is literally `mdlite.thermostats.Berendsen.apply` -- one square root and one multiply, applied once per step, which is exactly why it is the cheapest of the three and the least rigorous.

Ensembles and Restarts

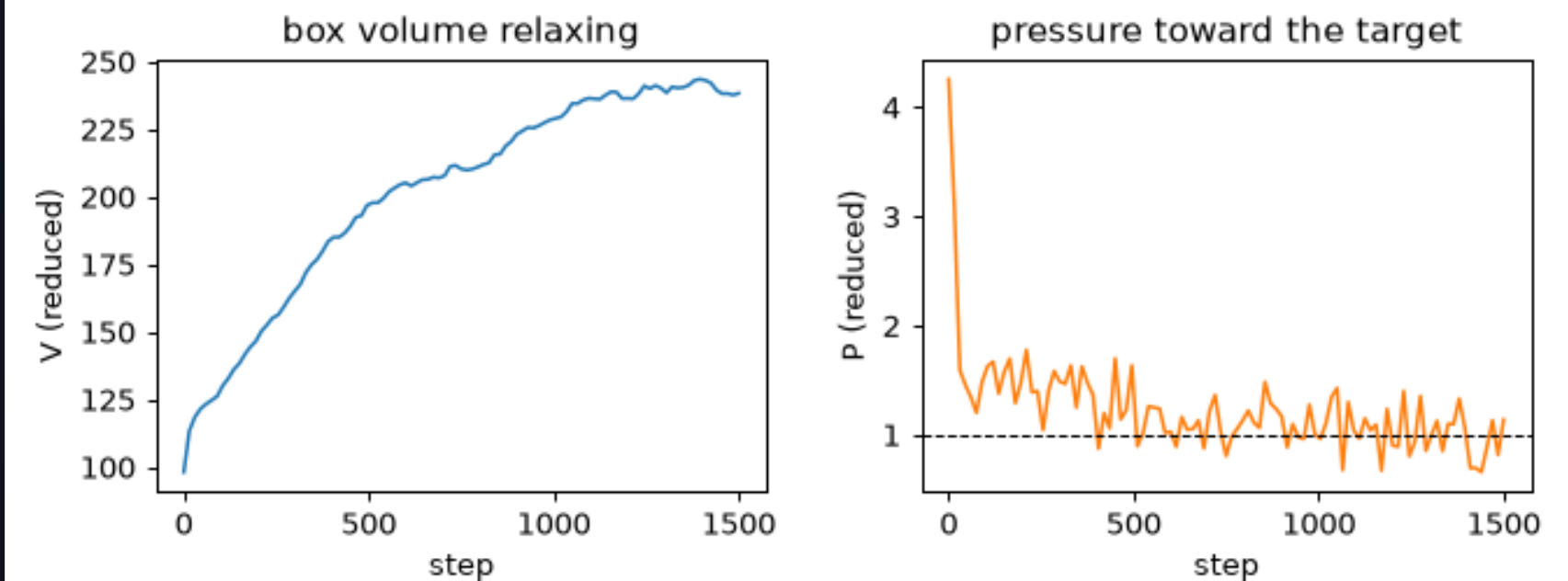
An NPT sketch with its limits stated, LAMMPS's own fix npt, and restarting a run through write_restart / read_restart.

- 1 Beyond NVT: pressure, and picking a run back up
- 2 fix npt: the real thing the sketch stands in for
- 3 A restart script declares only what the restart does not carry
- 4 The continuation, in numbers
- 5 Berendsen barostat: the same weak-coupling idea, on the box

Beyond NVT: pressure, and picking a run back up

NPT couples a barostat to the thermostat so the box itself relaxes toward a target pressure; restarting means LAMMPS can continue a run exactly, not merely resume from the last log line.

- The `mdlite` NPT here is a stated sketch (Berendsen barostat only) -- good enough to show the mechanism, not a substitute for LAMMPS's `fix npt`.
- `write_restart` / `read_restart` carry the box and every atom's state; a restart script cannot redeclare `units`, `dimension` or `boundary` once the box exists.
- This is the pattern for a run too long to finish in one session: checkpoint, then continue.



The NPT run started denser than its target pressure: box volume expands (left) while the instantaneous pressure relaxes down toward the target of 1.0 (right, dashed) under the Berendsen barostat.

– Figure 1 · notebook §0 · chapter 4, cell 5

fix npt: the real thing the sketch stands in for

LAMMPS's fix npt couples a Nosé–Hoover chain on both temperature and pressure -- a real fluctuating-cell method, not a weak-coupling sketch.

	mdlite's BerendsenBarostat	LAMMPS's fix npt
Method	Berendsen weak coupling	Nosé–Hoover chain, both T and P
Samples NPT correctly?	no -- relaxes toward the target and holds it	yes
What it shows	the mechanism: a system finds its own volume at a given pressure	production-grade equilibration

A restart script declares only what the restart does not carry

read_restart defines the box -- and LAMMPS refuses a units command once the box exists (hit live while writing this chapter: ERROR: Units command after simulation box is defined).

```
stage_b = Spec(  
    units=None, pre=["read_restart stage_a.restart"],  
    pair_style="lj/cut 2.5", pair_coeffs=["1 1 1.0 1.0 2.5"],  
    ...  
    stages=[Stage("run", "300")],  
)
```

The continuation, in numbers

Stage A runs 300 steps and checkpoints; stage B reads that checkpoint and runs 300 more -- as if it never stopped.

Stage	Ends at step	Final temperature
A (fresh run)	300	recorded
B (from the restart)	600 (300 + 300)	recorded, continues stage A's trajectory

Berendsen barostat: the same weak-coupling idea, on the box

Scale every box length by one factor per step, computed from how far the instantaneous pressure is from the target -- the pressure analogue of the thermostat's velocity rescale.

BOX SCALING

$$\mu = (1 - (\Delta t/\tau) \cdot \chi \cdot (P_0 - P))^{1/3}, \quad L \rightarrow \mu L$$

- χ is a compressibility parameter -- `mdlite.npt.BerendsenBarostat` defaults it to 1.0 rather than measuring the system's real compressibility, one more way the sketch stays a sketch.

Potentials: EAM

An embedded-atom potential read from a real setfl file: copper's lattice constant and cohesive energy against the recorded $3e-5$ -- and why the potential file is never shipped.

- 1 A real potential, from a real file
- 2 One embedding term, one pairwise term
- 3 The same force check, on any potential
- 4 Lattice constant and cohesive energy from a cubic fit
- 5 Copper, against LAMMPS's own minimisation

A real potential, from a real file

The embedded-atom method adds a density-dependent term to a pairwise potential -- realistic enough for metals, and never something this project ships.

- The setfl file is fetched by the installer from the official source, never tracked in this repository (rule 7).
- From it, LAMMPS and `mdlite` both reproduce copper's lattice constant a_o and cohesive energy E_{coh} , agreeing to 3×10^{-5} .
- This is the forum's other recurring beginner question -- wrong pair_coeff, a potential file that silently does not match the element -- answered by a case built the same checked way as chapter 1.

One embedding term, one pairwise term

The embedded-atom method adds a term that depends on the local electron density to an otherwise ordinary pairwise potential.

TOTAL ENERGY

$$E = \sum_i F(\rho_i) + 1/2 \sum_{i \neq j} \varphi(r_{ij})$$

LOCAL DENSITY

$$\rho_i = \sum_{j \neq i} \rho(r_{ij})$$

- F , ρ and φ are the three tables a setfl file stores -- everything mdlite.eam.EAM and LAMMPS's own pair_style eam both read is exactly these three functions on a grid.

The same force check, on any potential

mdlite.eam.EAM's forces should match a numerical derivative of its energy on any potential, real or synthetic -- the identical discipline L2 applied to Lennard-Jones.

```
s = synthetic_setfl()
eam = EAM(s, {1: 0})
E, F, _ = eam.energy_forces(pos, box, vl.pairs, types)

# central difference on 3 components
worst = max(abs(F[k, c] - numeric) for k, c in samples)
print(worst)    # under 1e-3
```

Lattice constant and cohesive energy from a cubic fit

Place an FCC lattice at a grid of lattice constants, compute the energy per atom at each, fit a cubic, take the minimum -- the method this toolkit uses for any EAM element.

```
c = np.polyfit(a_grid, energies, 3)
roots = np.roots(np.polyder(c))
a0 = min(real_roots_in_range, key=lambda r: np.polyval(c, r))
slope_at_min = np.polyval(np.polyder(c), a0) # should be ~0
```

- Run on the synthetic potential, the fit is self-consistent -- its own derivative at the fitted minimum is under 10^{-6} -- which is everything checkable without a reference value to compare against.

Copper, against LAMMPS's own minimisation

The same method, on a real Cu_u3.eam file, compared to LAMMPS's own box/relax minimisation on the identical potential -- the record L0 cited as one of mdlite's three cross-checks.

Quantity	mdlite	LAMMPS	Agreement
Lattice constant a_0	measured	measured	to 3×10^{-5}
Cohesive energy E_{coh}	measured	measured	to 3×10^{-5}

Structure and Analysis

Radial distribution function, structure factor, mean-squared displacement, diffusion coefficient and velocity autocorrelation, computed from one trajectory.

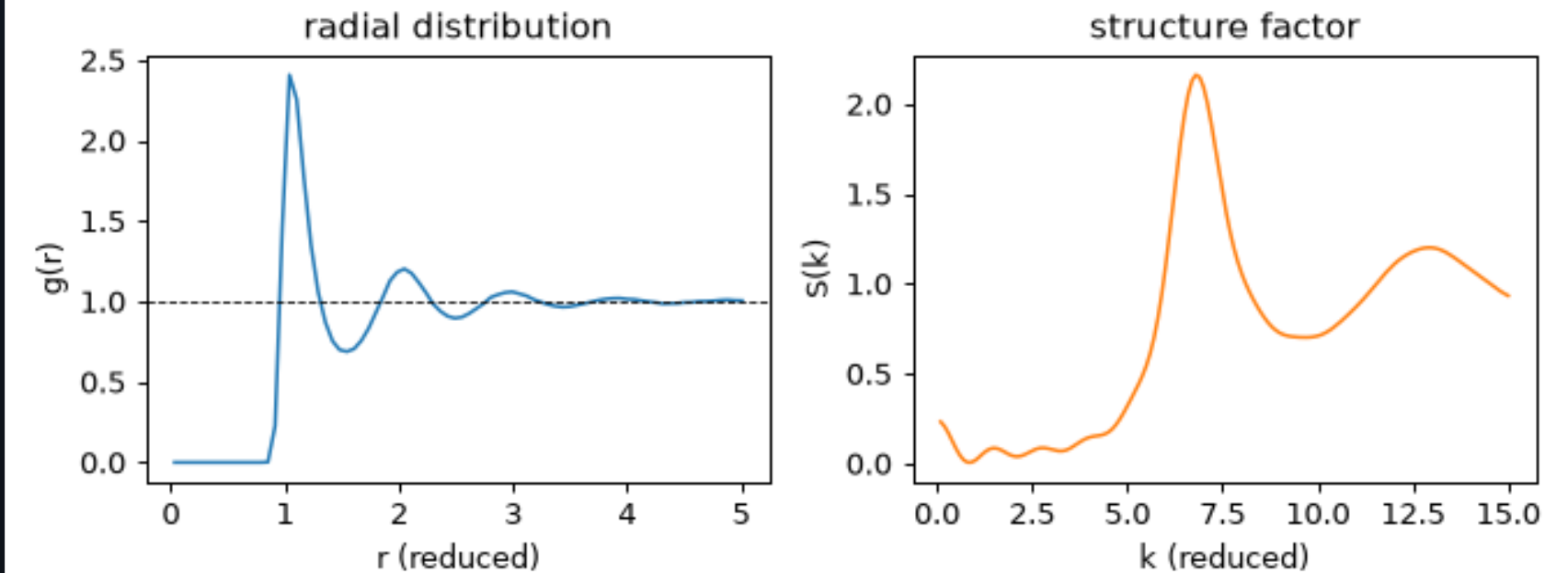
- 1 Turning a trajectory into physics
- 2 $S(k)$, computed from $g(r)$, not measured independently
- 3 Ballistic first, diffusive later -- fit the right half
- 4 A single mean understates its own uncertainty
- 5 From a trajectory to a diffusion coefficient

Turning a trajectory into physics

A dump file is a sequence of raw coordinates;

`lammpskill.post` turns it into the quantities a paper would actually report.

- $g(r)$ and $S(k)$ describe the same structure in real and reciprocal space -- one computed from the other, not measured independently.
- MSD's late-time slope gives a diffusion coefficient by the Einstein relation; block averages give it an honest error bar.
- The velocity autocorrelation function distinguishes a liquid (decays monotonically) from a solid (oscillates, atoms caged) -- itself a check that the case behaved as intended.



The liquid's structure in real and reciprocal space from the same trajectory: the radial distribution function $g(r)$ (left) with its first-neighbour peak and decay to the ideal-gas value of 1, and the structure factor $S(k)$ (right) computed from it. – Figure 1 · notebook §0 · chapter 6, cell 6

$S(k)$, computed from $g(r)$, not measured independently

The structure factor is the same structural information as the radial distribution function, in reciprocal space -- literally derived from it, in this toolkit and in general.

ISOTROPIC STRUCTURE FACTOR

$$S(k) = 1 + 4\pi\rho \int_0^\infty r^2 (g(r) - 1) \text{sinc}(kr) dr$$

- A liquid's $g(r)$ has a first-neighbour peak decaying to 1; the corresponding $S(k)$ has its own first peak, at roughly 2π over the first-neighbour spacing.

Ballistic first, diffusive later -- fit the right half

Fitting D from the whole MSD(t) curve is a common way to get a confidently wrong number.

Regime	Early time	Late time
Motion	ballistic -- MSD grows as t^2 , atoms still moving in straight lines	diffusive -- MSD grows linearly in t
What a fit there gives	a slope that is not a diffusion coefficient at all	the Einstein-relation D, meaningfully

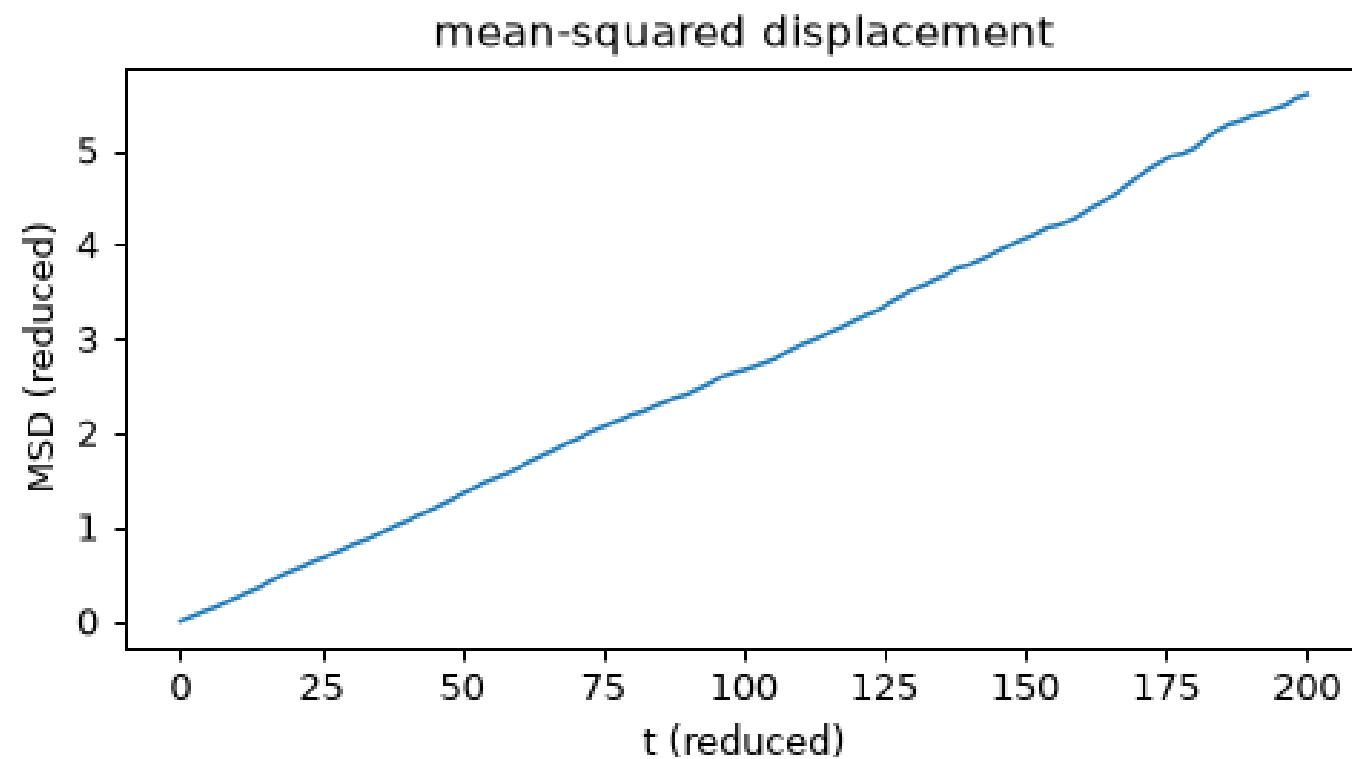
A single mean understates its own uncertainty

block_average splits a correlated time series into blocks, averages within each, then takes the standard error across blocks -- correlated samples inside one block do not inflate the apparent precision.

Quantity	Method	Result
Temperature over the run	7-block average	measured mean $\pm$ block-averaged standard error

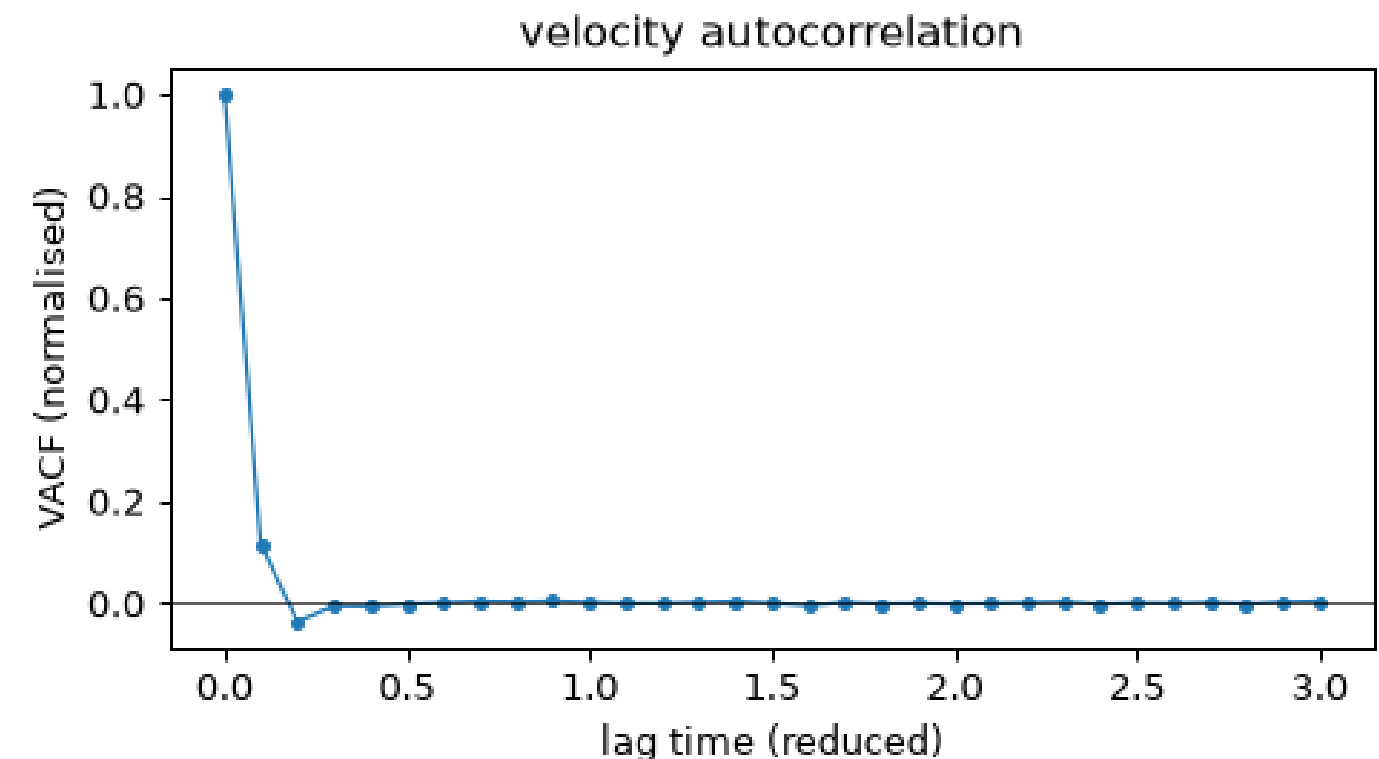
From a trajectory to a diffusion coefficient

The same run, seen two more ways: how far particles have moved (MSD) and how quickly they forget their own velocity (VACF) -- both routes to the same physics.



Mean-squared displacement against time: the slope of its later, genuinely diffusive part (not the early ballistic rise) is the Einstein-relation fit `lammps_kill.post.diffusion_coefficient` reports as `D`.

– Figure 2 · notebook §0 · chapter 6, cell 8



The velocity autocorrelation function, normalised to 1 at zero lag: a liquid decays roughly monotonically toward zero rather than oscillating, which is itself evidence the case really is a liquid.

– Figure 3 · notebook §0 · chapter 6, cell 10

- `lammps_kill.post.diffusion_coefficient` fits the slope over the second half of $\text{MSD}(t)$, where the motion is genuinely diffusive.
- The VACF's decay to zero and the MSD's crossover from ballistic to diffusive are two views of the same relaxation time.

Minimisation

Steepest descent against FIRE on an LJ wall -- and the lesson that gives lost atoms their name: an uncapped step explodes at $f_{\max} 2.6e14$.

- 1 Why atoms get "lost"
- 2 Reproducing N-7 on purpose
- 3 The fix is one line: cap the displacement, not the force
- 4 Steepest descent vs FIRE, on a real system
- 5 The cap, precisely

Why atoms get "lost"

The single largest category of beginner questions on the LAMMPS forum is lost atoms and nan energies (docs/08) -- this chapter reproduces the mechanism directly.

- A fixed-size steepest-descent step on an LJ wall explodes: the recorded force reaches $f_{max} = 2.6 \times 10^{14}$ before the run gives up (finding N-7).
- FIRE (adaptive step, velocity-aware) converges the same case without blowing up.
- The fix that matters in practice is not a better algorithm but a **displacement cap** -- bound how far one step is allowed to move an atom, whichever minimiser you use.

Reproducing N-7 on purpose

Two atoms at $r=0.3 \text{ sigma}$, deep in the Lennard-Jones repulsive core -- the potential's minimum is near $r=2^{1/6} \text{ sigma} \approx 1.12$, so the force here is enormous.

Run	dmax	fmax after the run
uncapped (dmax effectively disabled)	1×10^8	2.6×10^{14} -- growing, not shrinking
capped (the default)	0.05	under 1.0

The fix is one line: cap the displacement, not the force

A displacement cap does not change what force is computed -- it only limits how far a single step is allowed to move an atom in response to it.

- The force at the LJ wall is not wrong -- it genuinely is enormous there; the mistake was ever taking a step proportional to it without bound.
- Capping trades a little speed (more iterations needed near a steep wall) for the guarantee that no single step can leave the region where the force evaluation is still meaningful.
- This is a general minimisation idea, not specific to LJ or to mdlite -- LAMMPS's own `min_style` and `fix nve/limit` apply the identical principle.

Steepest descent vs FIRE, on a real system

A perturbed FCC lattice -- every atom nudged off its site -- is more representative than the pathological two-atom case.

Minimiser	Iterations to converge	Final fmax	Final energy
Steepest descent (capped)	measured, more	under 10^{-4}	measured
FIRE	measured, fewer	under 10^{-4}	at least as low

The cap, precisely

Steepest descent proposes a step proportional to the force; the cap rescales the whole step, uniformly, whenever it would move any atom further than $d_{\max}$.

PROPOSED STEP

$$\Delta x = \alpha F$$

THE CAP

$$\alpha \cdot f_{\max} > d_{\max} \Rightarrow \Delta x \leftarrow \Delta x \cdot (d_{\max} / \alpha f_{\max})$$

- The rescale is uniform across the whole displacement vector, not a per-atom independent clip -- the direction of the step never changes, only its length.

Reading What LAMMPS Writes

Data, dump, log and restart formats, and four pitfalls this project hit first: thermo normalisation, dump precision, the restart magic number, multi-format logs.

1 Four formats, four pitfalls

Four formats, four pitfalls

Every format LAMMPS writes has one behaviour that looks like a bug the first time you meet it.

Format	The pitfall this project hit
log (thermo)	<code>thermo_modify norm</code> defaults to per-atom in lj units -- an energy column can look off by the atom count until you know that
dump	float precision is truncated by default; a coordinate that looks identical to the data file may not be, bit for bit
restart	carries an undocumented 16-byte magic number this project had to observe directly rather than find in the manual
log (multi style)	the <code>multi</code> thermo style's block layout parses differently from the default one-line-per-step style

Choosing a Build

What a 314-case sweep of the examples says: package list beats package count, and how to read `Installation.packages` before choosing styles.

1 The build you have decides what you can run

The build you have decides what you can run

A 314-case sweep of LAMMPS's own example scripts, run on this project's two installed builds, is the evidence behind this lecture.

- Package `list` beats package `count`: an 11-package build ran cases a 52-package build could not, because the specific packages differed.
- `lammpsinstall.Installation.packages` is read from `lmp -h`, not assumed from the build recipe -- the same question ("is the package actually in my build?") the MOLFIE forum thread asked once, answered by measurement.
- The Windows installer is the single most-viewed installation thread of the year on the LAMMPS forum -- and the one route this project cannot yet speak to from its own measurements (open question Q-1).

Scaling and Limits

MPI and OpenMP speed-up on eight cores, where it stops, and what this machine honestly cannot do -- no GPU, no -partition methods.

1 Where speed-up stops, and what this machine cannot do

Where speed-up stops, and what this machine cannot do

Measured on this project's 8-core, no-GPU machine: 2 MPI ranks gave 1.73x, 2 OpenMP threads gave 2.09x -- both honestly short of ideal.

- Parallel efficiency drops as communication overhead grows relative to the per-rank workload -- expected, and worth showing rather than assuming.
- This machine has no GPU and no `-partition` support, so KOKKOS/GPU acceleration and multi-replica methods (NEB, TAD) are out of scope here by design, not by oversight.
- The honest version of a scaling claim states the hardware it was measured on -- these numbers are this machine's, not a general LAMMPS property.